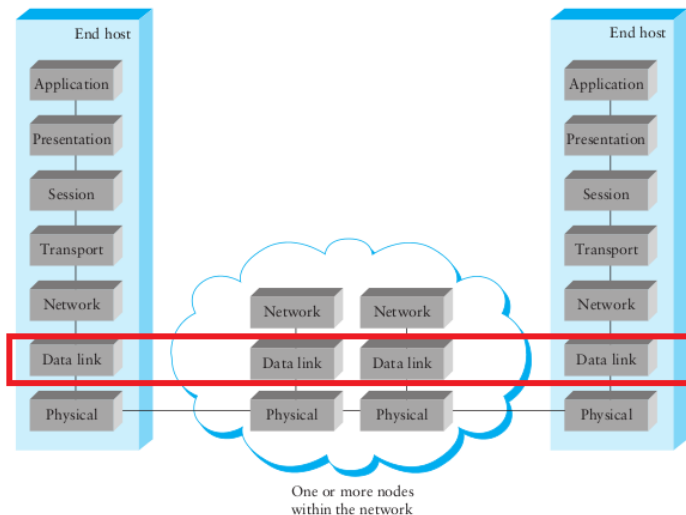


Eficiencia de Protocolos punto a punto

Redes de Comunicaciones y Cómputo Distribuido

25.08.2026

Arquitectura en capas



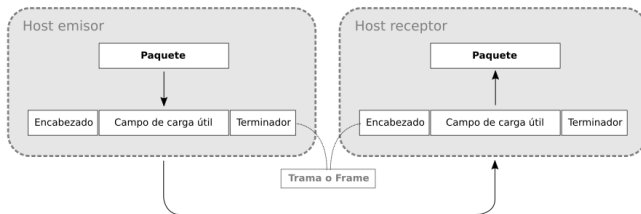
Las comunicaciones se dan en capas que se brindan servicios entre sí

Contexto:

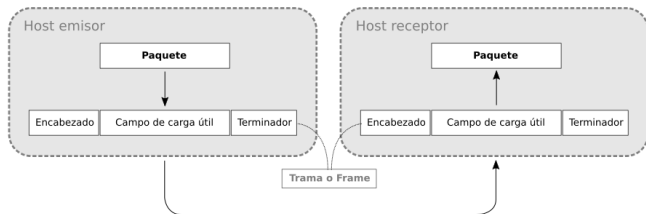
- **Caño serial:** No hay desorden.
- **Vulnerable a ruido impulsivo:** Lo que se recibe puede no ser lo que se envió (*error de transmisión*).

Problemas a resolver:

- **Codificación:** ¿Cómo representar los bits en el medio de transmisión?
- **Framing:** Encapsular los bits en frames agregando información de control para que puedan ser enviados.
- **Control y corrección de errores:** ¿Se produjo algún error?
¿Los podemos arreglar?
- **Transmisión confiable:** ¿Cómo logro fiabilidad si no puedo arreglar los errores? Solución a los frames descartados
- **Acceso al medio:** (*más adelante...*)



¿Qué es un frame?



¿Qué es un frame?

Mensaje intercambiado entre emisor y receptor que incluye los datos junto con la **información de control** necesaria para su interpretación (*encabezado*) y **verificación** (*checksum*).

¿Cómo se separan los frames en un tren de bits consecutivos?

- Largo fijo
- Largo en el encabezado
- Delimitadores con *bit-stuffing*

¿Cómo se separan los frames en un tren de bits consecutivos?

- Largo fijo
- Largo en el encabezado
- Delimitadores con *bit-stuffing*

Eficiencia de frame

$$\eta_{frame} = \frac{\text{largo de los datos}}{\text{largo total del frame}}$$

Calcule la eficiencia del frame tomando en cuenta solo el overhead impuesto por las siguientes técnicas de framing:

- a. Largo fijo
- b. Campo de 16 bits en el encabezado indicando el largo del frame
- c. Delimitadores de 8 bits usando *bit-stuffing*

Calcule la eficiencia del frame tomando en cuenta solo el overhead impuesto por las siguientes técnicas de framing:

- a. Para largo fijo tenemos que:

$$\eta_{\text{frame}} = \frac{\text{largo de los datos}}{\text{largo del frame}} = 1 \text{ los datos ocupan todo el frame.}$$

- b. Para campos en el encabezado tenemos que:

$$\begin{aligned}\eta_{\text{frame}} &= \frac{\text{largo del frame} - ?}{\text{largo del frame}} = \frac{\text{largo del frame} - |\text{campo}_{\text{size}}|}{\text{largo del frame}} \\ &= \frac{\text{largo del frame} - 16}{\text{largo del frame}}\end{aligned}$$

- c. Este último caso es variable ya que depende de la cantidad de bits que haya que incorporar al frame debido al stuffing.
Si n es dicha cantidad de bits, nos queda:

$$\eta_{\text{frame}} = \frac{\text{largo del frame} - 8 - n}{\text{largo del frame}}$$

- **Detección de errores:**

- ★ Bit de paridad
- ★ CRC (Cyclic Redundancy Check)
- ★ Checksum (Internet Checksum)
- ★ MD5 (Funciones criptográficas/Hash)

- **Corrección de errores (FEC):**

- ★ Hamming (Códigos de Hamming)
- ★ Reed-Solomon

Asegurarnos que las tramas son **entregadas en orden, sin errores y sin duplicados**, a pesar de los errores de transmisión.



Para implementar usamos una estrategia (a veces llamada Automatic Repeated reQuest - ARQ) basada en ACK y timeouts.

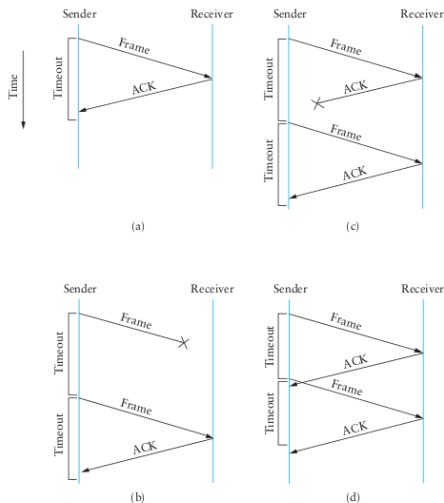
Asegurarnos que las tramas son **entregadas en orden, sin errores y sin duplicados**, a pesar de los errores de transmisión.



Para implementar usamos una estrategia (a veces llamada Automatic Repeated reQuest - ARQ) basada en ACK y timeouts.

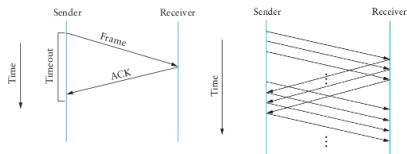
- **SWS=RWS=1**
- Surge la necesidad de secuenciar los frames para evitar el problema de las reencarnaciones
- ★ Para Stop & Wait se necesita poder secuenciar 2 frames.
- ★ Para Stop & Wait **bit requeridos = 1**.
 - Cada Frame debe ser reconocido por el receptor

Transmisión confiable: Stop and Wait



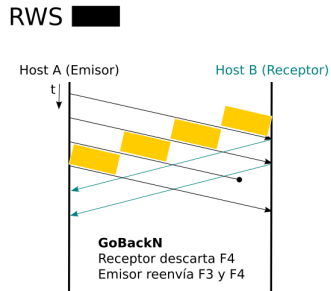
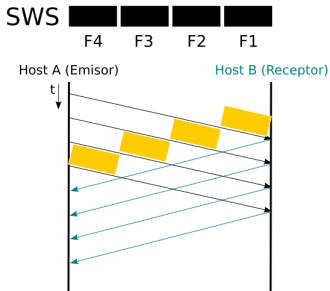
Si no usamos SEQ en c) y d) el receptor puede pensar que esta recibiendo una copia del frame inicial (reencarnación)

Motivación GENERAL: llenar el canal

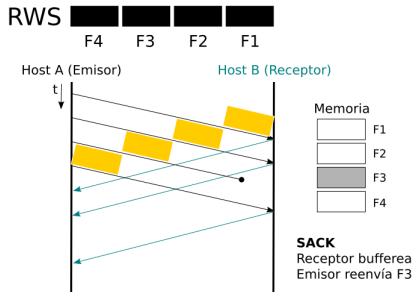
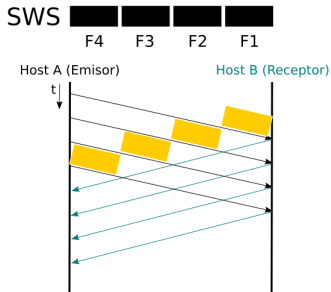


★ **Ventana de emisión óptima (maximiza la eficiencia del protocolo):** $SWS = V_{tx} * RTT / |Frame|$

Transmisión confiable: Sliding Window Go Back'N



Transmisión confiable: Sliding Window ACK Selectivo



Debemos aumentar la ventana de emisión para aprovechar mejor el canal:

$$\star SWS = V_{tx} * RTT / |Frame|$$

Si el receptor dispone de buffers, se puede usar SACK. Sino, no aporta a la eficiencia del protocolo.

$$\star RWS = \begin{cases} SWS & \text{Si hay SACK} \\ 1 & \text{Si no} \end{cases}$$

Y para distinguir reencarnaciones

$$\star \#frames \geq SWS + RWS$$

¿Cuántos bits hacen falta para secuenciar los frames?

$$\star \text{bits requeridos} = \lceil \log_2(SWS + RWS) \rceil$$

¿Cuánto tiempo se está transmitiendo con respecto al tiempo bloqueado esperando?

$$Eficiencia = \frac{T_{tx}(V)}{RTT(F)}$$

Con $T_{tx}(V)$ el tiempo de transmisión de una ventana y $RTT(F)$ el tiempo de ida y vuelta de un frame ****.

★ *Aumentar la eficiencia es estar bloqueado lo menos posible.*

**** **$RTT = 2 * Delay(F)$**

Diseñe posibles conjuntos de frames para los siguientes tipos de protocolos, asumiendo que se detectan errores usando CRC. (No hace falta aclarar el largo de los campos)

- a. Stop & Wait
- b. Sliding Window con GoBackN usando *Piggybacking*
- c. Sliding Window con ACK Selectivo

- a. Para el primer caso podríamos definir un frame para el emisor y otro para el receptor de la siguiente forma:

Emisor: *#SEQ(1bit); Datos; Checksum*

Receptor: *#ACK(1bit); Checksum*

- b. Para el segundo caso, al haber *Piggybacking* todos los frames pueden llegar a tener que cumplir simultáneamente los roles de emisión y recepción. Entonces proponemos un único frame como el siguiente:

#SEQ; #ACK; Datos; Checksum

- c. Para el último caso, a diferencia del punto anterior tenemos reconocimiento selectivo. Para estos casos se recomienda, por un tema de eficiencia, que el frame receptor reconozca los frames tanto acumulativa como individualmente. Entonces nos quedarían dos frames como los siguientes:

Emisor: *#SEQ; Datos; Checksum*

Receptor: *#ACK; #SACK; Checksum*

¿Qué pasa en cada uno de los casos anteriores, con respecto al tamaño de los frames del emisor y receptor?

¿Qué pasa en cada uno de los casos anteriores, con respecto al tamaño de los frames del emisor y receptor?

Los tamaños son diferentes en a) y c), aún cuando trabajamos con frames de tamaño fijo emisor y receptor son diferentes.

¿Qué pasa en cada uno de los casos anteriores, con respecto al tamaño de los frames del emisor y receptor?

Los tamaños son diferentes en a) y c), aún cuando trabajamos con frames de tamaño fijo emisor y receptor son diferentes.

Pueden ser iguales si usamos relleno. Por ejemplo, un receptor:

#ACK (16bits); #SACK (16bits); Padding; Checksum (16bits)

Un protocolo sobre un enlace punto a punto de 1Mbps y 0.25 segundos de delay, trabaja con ventana deslizante con GoBackN usando frames de largo fijo 2Kb y un CRC de 16bits para detectar errores.

- a. Calcule cuáles son los tamaños de ventana de emisión y recepción óptimos.
- b. ¿Cuántos bits hacen falta para secuenciar los frames?
- c. Calcule cuánto tiempo es necesario para transmitir 20Mb de datos asumiendo que no hay errores.

Calcular tiempo en Stop'nWait y GoBack'N-SACK

Calcular tiempo en Stop'nWait y GoBack'N-SACK

¿Qué no se puede olvidar?

1. Delay(F) no es igual a DELAY (salvo que se diga lo contrario)

Ejercicio 1

Un protocolo punto a punto trabaja sobre un enlace de 1Mbps y un delay de frame de 0.2 segundos. El protocolo usa un frame de largo fijo de 2Kb como el siguiente:

#SEQ (8bits); #ACK (8bits); #SACK (8bits); Datos; Checksum (16bits)

- ¿Qué porcentaje del delay del frame representa el tiempo de transmisión?
- Calcule la eficiencia del protocolo sobre el enlace.
- ¿Cuánto tiempo tardaría en enviarse 30Mb de datos?

2. DELAY se puede aproximar a T_{prop} , en enlaces de alta velocidad y frames pequeños

a. $SWS = V_{tx} * RTT / |Frame|$

donde $RTT = 2 * Delay(F) = 2(T_{tx}(F) + T_{prop})$

$RWS = 1$ para GoBackN.

b. ¿Cuántos bits hacen falta para secuenciar los frames?

$$\lceil \log_2(SWS + RWS) \rceil$$

c. Calcule cuánto tiempo es necesario para transmitir 20Mb de datos asumiendo que no hay errores.

$$|Datos| = |Frame| - |SEQ| - |Checksum| = X \text{ bits}$$

Entonces podemos calcular $\frac{20Mb}{Xb}$ frames.

Luego usamos un poco de Práctica 1,

$$Delay(\#frames) =$$

$$T_{tx}(\#frames) + T_{prop} = \frac{|frame| * \#frames}{1Mbps} + 0.25s$$

Un protocolo confiable punto a punto que usa sliding window, opera sobre un canal de 10 Mbps, usa SACK y un frame emisor de 5Kb como el siguiente:

#SEQ (8bits) ; Datos ; Checksum

- a. Proponga un frame para el receptor.
- b. ¿Cuál es el valor de delay para el cual el protocolo presenta un 100% de eficiencia?
- c. Si el delay fuera de 1 seg ¿Cuántos bits deberían ocupar los números de secuencia de manera de maximizar la eficiencia?



Frame propuesto:

#ACK (8bits); #SACK (8bits); Padding; Checksum

Un satélite artificial que será enviado para orbitar Marte usa un protocolo de ventana deslizante sobre un enlace de 60 Mbps. El satélite sólo envía datos y, desde la tierra, se usa un frame de reconocimiento de 1024 bits con la siguiente estructura:

#ACK (16bits); #SACK (16bits); Padding; Checksum (16bits)

- a. ¿Cuál es el tamaño de las ventanas de emisión y recepción?
- b. Proponga un frame del mismo largo para el satélite y calcule su eficiencia de frame.
- c. Dado que el delay a Marte es de 6 minutos, ¿cuál sería la eficiencia del protocolo asumiendo que no hay errores de transmisión?

Un protocolo punto a punto de capa de enlace usa ventana deslizante con un esquema de reconocimiento de acknowledge selectivo (SACK). Usa frames de largo fijo de 2 kbit y el campo de número de secuencia es de 4 bits.

- a. ¿Cuál es la máxima eficiencia que se puede alcanzar sobre un enlace de 10 kbps y 1 segundo de delay?
- b. En una conexión establecida usando este protocolo, el emisor envía los frames 4, 5, 6 y 7 seguidos, arribando con errores de CRC el frame 4 y el 6. Realice el diagrama de intercambio de paquetes que sucede entre emisor y receptor hasta recuperarse de ese error.

It is worth noting that reliability is a function that may be provided at the link level, but many modern link technologies omit this function. Furthermore, reliable delivery is frequently provided at higher levels, including both transport and, sometimes, the application layer. Exactly where it should be provided is a matter of some debate and depends on many factors. We describe the basics of reliable delivery here, since the principles are common across layers, but you should be aware that we are not just talking about a link-layer function.

- Capítulo 2 “Direct Link Networks”, Sección 2.3 (Framing) hasta Sección 2.5 (Reliable Transmission) inclusive - Computer Networks: A Systems Approach - Larry Peterson and Bruce Davie. Elsevier, 2012. License CC BY 4.0 (<https://github.com/SystemsApproach/book>)
- Capítulo 3 “La Capa de Enlace de Datos” - A.Tanenbaum.